

AIOT labs on Dragon Q6A SBC

The following laboratory setup uses two Radxa SBCs based on a **Qualcomm SoC**: the Radxa Dragon Q6A. The **Radxa Dragon Q6A** is a high-performance, low-power embedded computing platform powered by the **Qualcomm® Dragonwing™ QCS6490 processor**. It is designed for applications in **industrial IoT, edge AI, robotics, and smart terminal systems**.

The platform integrates advanced **AI acceleration capabilities**, high-speed interfaces, and comprehensive wireless connectivity, making it suitable for demanding embedded and real-time processing tasks. It supports a wide range of peripherals and communication protocols for edge computing applications.

The Radxa Dragon Q6A supports multiple operating systems, including **embedded and desktop Linux distributions**, as well as **Windows IoT**, providing flexibility for software development and deployment across different application domains.

This laboratory configuration employs **two Radxa Dragon Q6A boards** to implement distributed AI and communication workflows, leveraging the platform's processing power, networking capabilities, and support for edge-based inference applications.

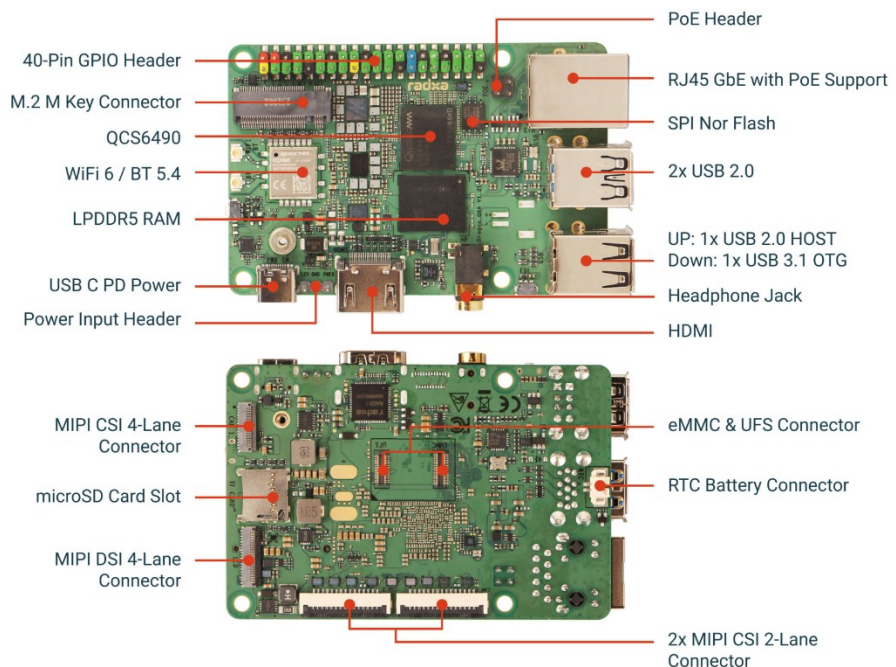


Fig 1 Radxa Dragon Q6A board

The Hardware

The board integrates Dragonwing **QCS6490 SoC** with Kryo cores

- 1×**Kryo Prime** @ 2.7 GHz + 3×Kryo Gold @ 2.4 GHz + 4×Kryo Silver @ 1.9 GHz

GPU: Qualcomm **Adreno™ 643** supporting:

- OpenGL® ES 3.2/2.0/1.1, Vulkan® 1.1/1.2/1.3
- OpenCL™ 2.2, DirectX Feature Level 12

DSP and AI Engine

- Compute Qualcomm Hexagon™ DSP with dual Hexagon Vector eXtensions(HVX)
- Additional Hexagon coprocessor 2.0 that complements the main CPU and DSP
- Hexagon Tensor Accelerator

RAM: LPDDR5 RAM 5500MT : **12GB**

Software Environment

The Radxa Dragon Q6A board operates under Linux Noble GNOME OS (R2).

Software development is performed primarily in **Python**, which is used for application integration, network communication, and processing logic.

The main software components used in this laboratory are:

- **whisper.cpp** – a lightweight speech-to-text (STT) engine used for speech transcription
- **piper** – a lightweight text-to-speech (TTS) engine used for speech synthesis

Initial Configuration

The initial configuration phase focuses on the installation, compilation, and validation of both **whisper.cpp** and **piper**. During this phase, all required software packages, libraries, and dependencies are installed to prepare the execution environment.

For **whisper.cpp**, both the standard and streaming versions are compiled and tested to ensure proper operation. The streaming version, **whisper-stream**, is particularly important for real-time speech processing applications.

In the next phase, **whisper-stream** is integrated with a Python application that filters the transcription output and generates UDP packets for transmission to the receiver node.

The filtering stage removes unwanted text fragments and formats the transcription before network transmission. **whisper.cpp** requires appropriate English speech recognition models to convert spoken audio into textual output.

Receiver Node Configuration

On the receiver side, the **piper** TTS application is installed and configured. A Python program is then developed to receive incoming UDP datagrams, extract the transmitted text, and forward it to **piper** for speech synthesis.

piper generates a synthesized audio stream using an English **female voice model**. The resulting audio output is redirected to the system audio player for playback through connected speakers or headphones.

Like `whisper.cpp`, `piper` also requires **language-specific voice models**, which must be installed to produce high-quality speech synthesis.

Overall, the software architecture enables a complete distributed speech processing pipeline, where spoken input is transcribed into text on one Radxa board and reconstructed as synthesized speech on another board through UDP-based communication.

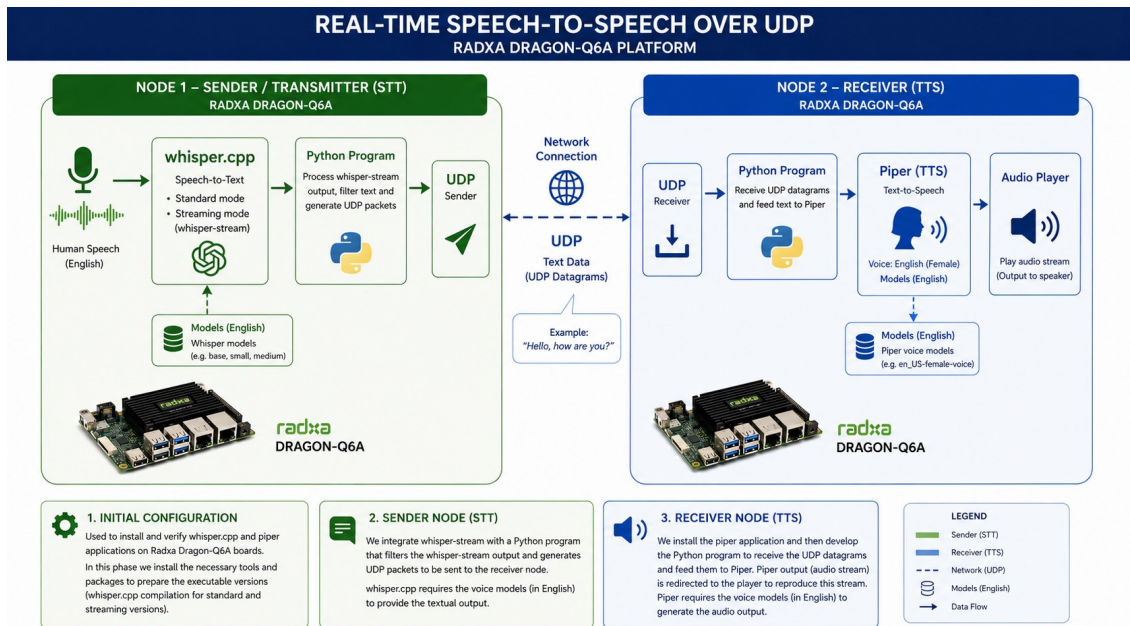


Fig 2. Two-Board Configuration for Speech-to-Text and Text-to-Speech Transmission

whisper.cpp installation

First, let's make sure your system is ready. Open a terminal on your Dragon Q6A and install the necessary build tools:

```
sudo apt update
sudo apt install build-essential cmake
```

Building from Source (Recommended)

This is the most direct method and is guaranteed to work on your board's ARM64 architecture.

Clone the repository:

```
git clone https://github.com/ggerganov/whisper.cpp.git
cd whisper.cpp
```

Compile the project: whisper server

```
cmake -B build
cmake --build build --config Release
```

To use `whisper.cpp` with a microphone on your Radxa Dragon Q6A board, you need to build the project's **dedicated "stream"** example. This version captures live audio and transcribes it in real-time.

First Install the SDL2 library: The streaming feature relies on the **SDL2** library to capture audio from your microphone. You can install it with this command:

```
sudo apt-get install libsdl2-dev
```

Build with SDL2 support: Now, you need to compile `whisper.cpp` with the `WHISPER_SDL2` flag enabled.

```
cmake -B build -DWHISPER_SDL2=ON
cmake --build build --config Release
```

Downloading speech models

The software requires a model to perform transcriptions. The `base.en` model is a good starting point for English. You can download other models later depending on your needs for speed and accuracy.

```
bash ./models/download-ggml-model.sh small.en
bash ./models/download-ggml-model.sh tiny.en
```

Running the Real-Time Transcription

Before you run the program, it's a good idea to verify your microphone setup.

Identify Your Microphone: List all recording devices to find the correct ID for your mic.

```
arecord -l
```

The output will show you a **list of devices** with their card and device numbers. You will use the card number in the next step.

- **Run the Streaming Application:** Once you know your microphone's card number, you can start the real-time transcription. Use the `stream` or `whisper-stream` command to run it. Here's an example using card 0 and the `base.en` model:

```
./build/bin/whisper-stream -m ./models/ggml-base.en.bin -c -t 6 --step 2048 --length 2048
```

If the name of the executable is slightly different, you can try `./build/bin/stream`.

Here's what the key parameters do:

- c 0: Selects the audio capture device with card number 0. Adjust this if your microphone has a different number.
- step 2048: Sets the time between processing steps to 2048 milliseconds.
- length 2048: Sets the total audio length to process in one window to 2048 milliseconds.

Once the program is running, you should be able to speak into your microphone and see the transcription appear in the terminal. The program will stop by pressing `Ctrl+C` and will then show you some performance timings.

Filtering and sending text over UDP

Here's a Python program that captures the text output from `whisper-stream`, filters and sends it over the internet using sockets. The program runs the streaming process in the background, reads its output line by line, and transmits each transcribed segment to a remote server.

Captures text from `whisper-stream` and sends each line as a UDP datagram.

Includes filters for unwanted content.

"""

```
import subprocess
import socket
import argparse
import signal
import sys
import re
from datetime import datetime

def clean_transcription(text):
    """Remove unwanted elements from transcription text."""
    # Remove IP addresses in brackets: [192.168.1.100]
    text = re.sub(r'\[\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\]', '', text)
    # Remove [BLANK_AUDIO] tag
    text = re.sub(r'\[BLANK_AUDIO\]', '', text, flags=re.IGNORECASE)
    # Remove action descriptions in brackets: [Sighs], [Laughs], [Music], etc.
    text = re.sub(r'\[[A-Za-z\s]+\]', '', text)
    # Remove parenthetical comments like (dramatic music), (applause), etc.
    text = re.sub(r'\([^\)]*\)', '', text)
    # Remove any remaining empty brackets
    text = re.sub(r'\[\]', '', text)
    # Clean up extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def main():
    parser = argparse.ArgumentParser(description="Send whisper.cpp transcriptions via UDP")
    parser.add_argument("--host", required=True, help="Destination IP address")
    parser.add_argument("--port", type=int, required=True, help="Destination port")
    parser.add_argument("--model", default="./models/ggml-tiny.en.bin", help="Whisper model path")
    parser.add_argument("--threads", "-t", type=int, default=6, help="Number of threads")
    parser.add_argument("--step", type=int, default=2048, help="Step size in ms")
    parser.add_argument("--length", type=int, default=2048, help="Audio length in ms")
    parser.add_argument("--whisper-path", default="./build/bin/whisper-stream", help="Path to whisper-stream executable")
    parser.add_argument("--no-filter", action="store_true", help="Disable content filtering (send raw output)")

    args = parser.parse_args()
    # Create UDP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    dest = (args.host, args.port)
    print(f" * 60)
    print(f" 🎙️ Whisper UDP Sender")
    print(f" Destination: {args.host}:{args.port}")
    print(f" Model: {args.model}")
    print(f" Threads: {args.threads}")
```

```

print(f"    Step/Length: {args.step}ms / {args.length}ms")
print(f"    Filtering: {'Disabled' if args.no_filter else 'Enabled'}")
print(f"=" * 60)
# Whisper-stream command
cmd = [
    args.whisper_path,
    "-m", args.model,
    "-t", str(args.threads),
    "--step", str(args.step),
    "--length", str(args.length)
]

print(f"\nRunning: {' '.join(cmd)}\n")
# Cleanup handler
def cleanup(signum, frame):
    print("\nStopping...")
    if process and process.poll() is None:
        process.terminate()
    sock.close()
    sys.exit(0)

signal.signal(signal.SIGINT, cleanup)
signal.signal(signal.SIGTERM, cleanup)
# Launch whisper-stream
try:
    process = subprocess.Popen(
        cmd,
        stdout=subprocess.PIPE,
        stderr=subprocess.DEVNULL,
        text=True,
        bufsize=1
    )
except FileNotFoundError:
    print(f"Error: '{args.whisper_path}' not found. Build whisper.cpp with SDL2 support first.")
    sys.exit(1)

print("\n👂 Listening... Speak into the microphone (Ctrl+C to stop)\n")
# Read lines, filter, display locally, and send as UDP
for line in process.stdout:
    line = line.strip()
    if line:
        # Strip ANSI escape codes
        line = re.sub(r'\x1b\[0-9;]*[a-zA-Z]', '', line)
        # Apply filters (unless disabled)
        if args.no_filter:
            clean_line = line
        else:
            clean_line = clean_transcription(line)
        # Only print and send if there's content after filtering
        if clean_line:
            timestamp = datetime.now().strftime("%H:%M:%S")
            # Show both raw and filtered in terminal
            if line != clean_line:
                print(f" [{timestamp}] 🟡 Raw: {line}")
                print(f" [{timestamp}] 🟢 Filtered: {clean_line}")
            else:
                print(f" [{timestamp}] 📝 {clean_line}")
            # Send the filtered version via UDP
            sock.sendto(clean_line.encode("utf-8"), dest)

cleanup(None, None)

if __name__ == "__main__":
    main()

```

Rending text sent over UDP

The following program receives UDP packets containing textual data and displays the received text messages. It may also append the sender's IP address for identification and debugging purposes.

```
-----
#!/usr/bin/env python3
"""
whisper_udp_receiver.py
Receives UDP datagrams from whisper_udp_sender.py and prints only the message.
"""
import socket
import argparse
import signal
import sys
from datetime import datetime

def main():
    parser = argparse.ArgumentParser(description="Receive whisper transcriptions via UDP")
    parser.add_argument("--port", type=int, default=9999, help="Port to listen on")
    parser.add_argument("--show-ip", action="store_true", help="Show sender IP address with each message")
    parser.add_argument("--show-timestamp", action="store_true", help="Show local timestamp with each message")
    args = parser.parse_args()
    # Create and bind UDP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(("0.0.0.0", args.port))

    def cleanup(signum, frame):
        print("\nStopping receiver...")
        sock.close()
        sys.exit(0)

    signal.signal(signal.SIGINT, cleanup)
    signal.signal(signal.SIGTERM, cleanup)
    print(f"🔊 Listening for transcriptions on port {args.port}... (Ctrl+C to stop)")
    print("-" * 60)
    while True:
        try:
            data, addr = sock.recvfrom(65535)
            text = data.decode("utf-8")
            # Build the output line based on options
            output_parts = []
            if args.show_timestamp:
                output_parts.append(f"[{datetime.now().strftime('%H:%M:%S')}]")
            if args.show_ip:
                output_parts.append(f"[{addr[0]}]")
            output_parts.append(text)
            # Print the message
            print(" ".join(output_parts))

        except Exception as e:
            print(f"Error: {e}")
            break

if __name__ == "__main__":
    main()
-----
```

Usage (shell) version:

```
$ ./whisper_udp_rcv.py --port 8064 🔊 Listening for transcriptions on port 8064...
(Ctrl+C to stop)
```

Installing piper

piper application (TTS) may be installed with :

```
pip install --break-system-packages piper-tts
```

To run the piper at any place we need to add in `~/ .bashrc`

```
PATH=$PATH:/home/rock/.local/bin
```

Then run

```
source .bashrc
```

Now we can continue the work in `whisper.cpp` directory and add voice models to `models` directory.

Downloading voice files

Download the model file

```
wget https://huggingface.co/rhasspy/piper-voices/resolve/main/en/en_GB/southern_english_female/low/en_GB-southern_english_female-low.onnx
```

Download the configuration file

```
wget https://huggingface.co/rhasspy/piper-voices/resolve/main/en/en_GB/southern_english_female/low/en_GB-southern_english_female-low.onnx.json
```

We can rename these files as :

```
mv en_GB-southern_english_female-low.onnx female.onnx
mv en_GB-southern_english_female-low.onnx.json female.onnx.json
```

```
:~/whisper.cpp$ ls models
```

```
convert-h5-to-coreml.py      for-tests-ggml-medium.en.bin
convert-h5-to-ggml.py        for-tests-ggml-small.bin
convert-pt-to-ggml.py        for-tests-ggml-small.en.bin
convert-silero-vad-to-ggml.py for-tests-ggml-tiny.bin
convert-whisper-to-coreml.py for-tests-ggml-tiny.en.bin
convert-whisper-to-openvino.py for-tests-silero-v6.2.0-ggml.bin
download-coreml-model.sh     generate-coreml-interface.sh
download-ggml-model.cmd      generate-coreml-model.sh
download-ggml-model.sh       ggml-small.en.bin
download-vad-model.cmd       ggml-tiny.en.bin
download-vad-model.sh        ggml_to_pt.py
female.onnx                  Llama-3.2-1B-Instruct-Q4_0.gguf
female.onnx.json             Llama-3.2-3B-Instruct-Q4_0.gguf
for-tests-ggml-base.bin      README.md
for-tests-ggml-base.en.bin   requirements-coreml.txt
for-tests-ggml-large.bin     requirements-openvino.txt
for-tests-ggml-medium.bin
```

```
rock@radxa-dragon-q6a:~/whisper.cpp$
```

Receiving text sent over UDP and activating piper

The following program integrates the UDP receiver program with `piper` and `aplayer`.

```
-----
#!/usr/bin/env python3
"""
whisper_udp_receiver.py
Receives UDP datagrams from whisper_udp_sender.py, prints the message,
and speaks it aloud using Piper TTS.
"""

import socket
import argparse
import signal
import sys
import subprocess
import os
from datetime import datetime

def main():
    parser = argparse.ArgumentParser(description="Receive whisper transcriptions via UDP and speak them with Piper")
    parser.add_argument("--port", type=int, default=9999, help="Port to listen on")
    parser.add_argument("--show-ip", action="store_true", help="Show sender IP address with each message")
    parser.add_argument("--show-timestamp", action="store_true", help="Show local timestamp with each message")
    parser.add_argument("--piper-model", default="models/female.onnx", help="Piper TTS model file path")
    parser.add_argument("--no-speak", action="store_true", help="Disable text-to-speech output (print only)")
    parser.add_argument("--quiet", action="store_true", help="Suppress Piper ONNX runtime warnings")
    args = parser.parse_args()

    # Verify piper is installed
    if not args.no_speak:
        if not os.path.exists(args.piper_model):
            print(f"Warning: Piper model '{args.piper_model}' not found.")
            print("Download it first or use --no-speak to disable TTS.")
            print("Continuing in print-only mode...\n")
            args.no_speak = True
        else:
            # Check if piper command is available
            try:
                subprocess.run(["piper", "--help"], capture_output=True, check=True)
            except (subprocess.CalledProcessError, FileNotFoundError):
                print("Warning: 'piper' command not found. Install piper-tts first.")
                print("Continuing in print-only mode...\n")
                args.no_speak = True

    # Create and bind UDP socket
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(("0.0.0.0", args.port))

    def cleanup(signum, frame):
        print("\nStopping receiver...")
        sock.close()
        sys.exit(0)

    signal.signal(signal.SIGINT, cleanup)
    signal.signal(signal.SIGTERM, cleanup)
    print(f"👂 Listening for transcriptions on port {args.port}... (Ctrl+C to stop)")
    if not args.no_speak:
        print(f"🗣 Speaking with Piper model: {args.piper_model}")
    print("-" * 60)
    while True:
        try:
```

```

data, addr = sock.recvfrom(65535)
text = data.decode("utf-8")
# Skip empty messages
if not text.strip():
    continue
# Build the output line based on options
output_parts = []
if args.show_timestamp:
    output_parts.append(f"[{datetime.now().strftime('%H:%M:%S')}]")
if args.show_ip:
    output_parts.append(f"[{addr[0]}]")
output_parts.append(text)
# Print the message
print(" ".join(output_parts))
# Speak the message with Piper (unless disabled)
if not args.no_speak:
    try:
        # Build the piper command
        piper_cmd = ["piper", "--model", args.piper_model, "--output-raw"]
        # Suppress stderr to hide ONNX runtime warnings
        stderr_target = subprocess.DEVNULL if args.quiet else None
        # Run piper and pipe to aplay for audio output
        piper_process = subprocess.Popen(
            piper_cmd,
            stdin=subprocess.PIPE,
            stdout=subprocess.PIPE,
            stderr=stderr_target
        )
        aplay_process = subprocess.Popen(
            ["aplay", "-r", "16000", "-f", "S16_LE", "-t", "raw", "-c", "1"],
            stdin=piper_process.stdout,
            stdout=subprocess.DEVNULL,
            stderr=subprocess.DEVNULL
        )
        # Send the text to piper
        piper_process.stdin.write(text.encode("utf-8"))
        piper_process.stdin.close()
        # Wait for both processes to finish
        piper_process.wait()
        aplay_process.wait()
    except Exception as e:
        print(f" ⚠️ TTS error: {e}")

except Exception as e:
    print(f"Error: {e}")
    break

if __name__ == "__main__":
    main()

```

Usage (shell) version:

```
$ ./whisper_udp_recv_piper.py --port 8064 --piper-model models/female.onnx
```

Usage Examples

Basic usage (speaks with your Southern English female voice):

```
./whisper_udp_receiver_piper.py --port 9999
```

Use a different model:

```
./whisper_udp_receiver_piper.py --port 9999 --piper-model models/female.onnx
```

Print only (no speech):

```
./whisper_udp_receiver_piper.py --port 9999 --no-speak
```

Hide ONNX runtime warnings:

```
./whisper_udp_receiver_piper.py --port 9999 --quiet
```

Show timestamps with spoken output:

```
./whisper_udp_receiver_piper.py --port 9999 --show-timestamp
```

Notes

- The audio plays through **aplay** with 16000 Hz sample rate, which matches most Piper models. If your voice model uses a different sample rate, adjust the **-r** value in the **aplay** command.
- Each message is spoken as it arrives. If messages arrive faster than they can be spoken, they will queue up.
- If you need to stop speaking immediately, press **Ctrl+C** twice.

Model	Size	Notes
tiny.en	~75 MiB	Very fast, but lower accuracy. A good choice for initial tests.
base.en	~142 MiB	A good balance between speed and accuracy for English.
small.en	~466 MiB	Significantly better accuracy, still manageable for many systems.
medium.en	~1.5 GiB	Excellent accuracy but requires more system resources and time.

llama.cpp and LLMs

llama.cpp is an open-source software project that provides a lightweight, high-performance way to run **large language models** (LLMs) locally on consumer hardware. It was originally created to demonstrate that modern LLMs could be executed efficiently using standard CPUs (and optionally GPUs) without requiring heavy cloud infrastructure.

1. Core Role of llama.cpp

The main role of llama.cpp is **local inference for transformer-based language models**, especially Meta's LLaMA family and compatible models.

In practical terms, it acts as a bridge between:

- **Model files** (usually in GGUF format)
- **Your local machine (CPU/GPU)**
- **Text generation tasks**

Its purpose is to let users run chatbots, text generators, coding assistants, and reasoning models **without needing an API or cloud service**.

2. Key Functions

2.1 Model Inference Engine

At its core, llama.cpp performs **inference**, meaning:

- It takes input text (a prompt)
- Processes it through a neural network
- Produces generated text output token-by-token

It supports optimized inference for transformer architectures used in modern LLMs.

2.2 CPU-Optimized Execution

One of its defining features is efficient CPU usage:

- Uses SIMD optimizations (AVX, AVX2, AVX-512, **NEON** on ARM)
- Minimizes memory overhead
- Allows usable performance even without a GPU

This makes it possible to run **7B-70B** parameter models on personal computers, depending on quantization.

2.3 Quantization Support

It supports **quantized models**, which reduce model size and memory usage by storing weights in lower precision formats (e.g., 4-bit, 5-bit, 8-bit).

Benefits:

- Lower RAM usage
- Faster inference on CPU
- Trade-off: slight reduction in model quality

This is essential for running large models on consumer hardware.

2.4 GGUF Model Format Support

`llama.cpp` primarily uses the **GGUF format**, which:

- Stores model weights, tokenizer data, and metadata
- Is optimized for fast loading and inference
- Supports multiple quantization levels

GGUF replaced earlier formats like GGML for better extensibility.

2.5 GPU Acceleration (Optional)

While CPU-first, `llama.cpp` also supports:

- CUDA (NVIDIA GPUs)
- Metal (Apple Silicon GPUs)
- Vulkan (cross-platform GPU backend)

This allows hybrid execution where parts of the model run on GPU for speed gains.

2.6 Chat and Prompt Interfaces

It provides:

- Command-line chat interface
- Server mode (REST API-like endpoints)
- Integration support for applications and bots

This makes it usable both for developers and end-user tools.

2.7 Streaming Token Output

Instead of waiting for full responses, it:

- Generates text token-by-token
- Streams output in real time

This improves interactivity for chat applications.

3. Typical Use Cases

- Local AI **chatbots** (privacy-focused)
- Offline coding assistants
- Research and experimentation with LLMs
- Running models on edge devices (laptops, Raspberry Pi-class systems)
- Educational exploration of transformer inference

llama.cpp build

Let us build `llama.cpp` with the Correct Backend

The **OpenCL backend is the recommended path for GPU acceleration** on this hardware.

We will need to compile `llama.cpp` from source, specifically enabling the OpenCL backend

Clone the repository

```
git clone https://github.com/ggml-org/llama.cpp
cd llama.cpp
```

Install prerequisites

```
sudo apt update
sudo apt install cmake build-essential
sudo apt install ocl-icd-opencl-dev opencl-headers clinfo
clinfo
```

Configure and build with Cmake

This command creates a build directory and enables the OpenCL backend.

```
mkdir build
cd build
cmake .. -DGGML_OPENCL=ON
cmake --build . --config Release -j $(nproc)
```

Download a Verified Model

A known, compatible source for the **Llama 3.2 3B** model is a repository by **muke1**, which provides a high-quality pure **Q4_0** version.

You can download it directly using this command on your board:

```
curl -L -O https://huggingface.co/mukel/Llama-3.2-3B-Instruct-GGUF/resolve/main/Llama-3.2-3B-Instruct-Q4\_0.gguf
```

```
curl -L -O https://huggingface.co/mukel/Llama-3.2-1B-Instruct-GGUF/resolve/main/Llama-3.2-1B-Instruct-Q4_0.gguf
```

This file is ready to use and should work very well with your setup.

Run Your Model

Once you have your **Q4_0** GGUF file, you can run it using the `llama-cli` tool you built, as we discussed before.

A basic command looks like this:

```
build/bin/llama-cli -m ./Llama-3.2-3B-Instruct-Q4_0.gguf
```

Using piper with whisper-talk-llama

Let us present the complete command to run all components together :

```
build/bin/whisper-talk-llama -mw models/ggml-tiny.en.bin \
-m1 models/Llama-3.2-1B-Instruct-Q4_0.gguf -t 4 --speak ./speak 2>/dev/null
```

Explanation

The above command line creates a **complete offline voice assistant** that listens to your speech, generates AI responses, and speaks them back to you, all running locally on our Radxa Dragon Q6A board.

Element	Details
build/bin/whisper-talk-llama	The main executable that orchestrates the entire pipeline (voice → text → AI → speech)
-mw models/ggml-tiny.en.bin	Small English speech recognition model (42MB). Transcribes your spoken words into text. " tiny " is fastest but least accurate; you can upgrade to "base" or "small" for better quality
-m1 models/Llama-3.2-1B-Instruct-Q4_0.gguf	The AI language model (1 billion parameters, 4-bit quantized). Generates intelligent responses to your transcribed speech. Q4_0 quantization balances quality and memory usage
-t 4	Uses 4 CPU threads for parallel processing. Match this to your board's available cores for optimal performance
--speak ./speak	Enables text-to-speech output and tells the program to call ./speak (a script in your current directory) to convert AI text responses into audible speech
2>/dev/null	Hides harmless warning messages (like the ONNX Runtime GPU detection message) by redirecting stderr to nowhere , keeping your terminal output clean

The Speak Script (./speak)

The **--speak** flag requires an external script that handles the final text-to-speech step. The script reads from **./examples/talk-llama/to_speak.txt** (where **whisper-talk-llama** writes the AI's response) and pipes it to **piper**:

```
-----
#!/bin/bash
# Read the text from the file that whisper-talk-llama writes to
SPEAK_FILE="/home/rock/whisper.cpp/examples/talk-llama/to_speak.txt"
# Wait a moment for the file to be written
sleep 0.5

if [ -f "$SPEAK_FILE" ]; then
    TEXT=$(cat "$SPEAK_FILE")
    if [ -n "$TEXT" ]; then
        echo "Speaking: $TEXT"
        echo "$TEXT" | piper --model models/female.onnx --output-raw | \
        aplay -q -r 16000 -f S16_LE -t raw -
    fi
else
    echo "No text file found at $SPEAK_FILE"
fi
-----
```

The following Python program is the transcription of the shell script into Python program using the `subprocess` to each executable component.

```
-----
#!/usr/bin/env python3
"""
Voice Assistant: whisper-talk-llama + Piper TTS
Equivalent to: build/bin/whisper-talk-llama -mw models/ggml-tiny.en.bin
               -ml models/Llama-3.2-1B-Instruct-Q4_0.gguf -t 4 --speak ./speak
2>/dev/null
"""
import subprocess
import sys
import os
# --- Configuration ---
WHISPER_TALK_PATH = os.path.expanduser("~/whisper.cpp/build/bin/whisper-talk-llama")
WHISPER_MODEL = os.path.expanduser("~/whisper.cpp/models/ggml-tiny.en.bin")
LLAMA_MODEL = os.path.expanduser("~/whisper.cpp/models/Llama-3.2-1B-Instruct-
Q4_0.gguf")
THREADS = 4

def run_voice_assistant():
    """Launch the complete voice assistant pipeline."""
    # Build the command exactly as you run it in the terminal
    cmd = [
        WHISPER_TALK_PATH,
        "-mw", WHISPER_MODEL,
        "-ml", LLAMA_MODEL,
        "-t", str(THREADS),
        "--speak", "./speak"
    ]

    print("🔊 Starting Voice Assistant")
    print("=" * 50)
    print(f"Whisper model: {os.path.basename(WHISPER_MODEL)}")
    print(f"Llama model: {os.path.basename(LLAMA_MODEL)}")
    print(f"CPU threads: {THREADS}")
    print(f"TTS engine: Piper (female.onnx)")
    print("=" * 50)
    print("Speak into your microphone. The AI will respond with voice.")
    print("Press Ctrl+C to exit.\n")
    try:
        # Run the process
        # stderr=subprocess.DEVNULL silences warnings (equivalent to 2>/dev/null)
        process = subprocess.run(
            cmd,
            stderr=subprocess.DEVNULL, # Hide ONNX Runtime warnings
            cwd=os.path.expanduser("~/whisper.cpp") # Run from whisper.cpp directory
        )
        # Exit with the same code as the subprocess
        sys.exit(process.returncode)
    except KeyboardInterrupt:
        print("\n\n👋 Voice assistant stopped.")
        sys.exit(0)
    except FileNotFoundError:
        print(f"❌ ERROR: {WHISPER_TALK_PATH} not found!")
        print("Make sure whisper-talk-llama is built correctly.")
        print("Expected location: ~/whisper.cpp/build/bin/whisper-talk-llama")
        sys.exit(1)

if __name__ == "__main__":
    run_voice_assistant()
-----
```

The following is the rewritten Python program with **command-line arguments** for the **Whisper** model, **Llama** model, and thread count. We can now change these settings without editing the code.

```
-----
#!/usr/bin/env python3

import subprocess
import sys
import os
import argparse

# --- Default Configuration ---
WHISPER_TALK_PATH = os.path.expanduser("~/whisper.cpp/build/bin/whisper-talk-llama")
DEFAULT_WHISPER_MODEL = "ggml-tiny.en.bin"
DEFAULT_LLAMA_MODEL = "Llama-3.2-1B-Instruct-Q4_0.gguf"
DEFAULT_THREADS = 4
MODELS_DIR = os.path.expanduser("~/whisper.cpp/models")

def parse_arguments():
    parser.add_argument(
        "-w", "--whisper",
        type=str,
        default=DEFAULT_WHISPER_MODEL,
        help=f"Whisper model filename (default: {DEFAULT_WHISPER_MODEL})"
    )
    parser.add_argument(
        "-l", "--llama",
        type=str,
        default=DEFAULT_LLAMA_MODEL,
        help=f"Llama model filename (default: {DEFAULT_LLAMA_MODEL})"
    )
    parser.add_argument(
        "-t", "--threads",
        type=int,
        default=DEFAULT_THREADS,
        help=f"Number of CPU threads (default: {DEFAULT_THREADS})"
    )
    parser.add_argument(
        "--list-models",
        action="store_true",
        help="List available models in the models directory and exit"
    )
    return parser.parse_args()

def list_available_models():
    """Display available Whisper and Llama models in the models directory."""
    print(f"\n Models directory: {MODELS_DIR}")
    print("=" * 60)
    if not os.path.exists(MODELS_DIR):
        print(" Models directory not found!")
        return
    # List all model files
    models = sorted(os.listdir(MODELS_DIR))
    whisper_models = [m for m in models if m.startswith("ggml-") and
m.endswith(".bin")]
    llama_models = [m for m in models if m.endswith(".gguf")]
    other_files = [m for m in models if m not in whisper_models and m not in
llama_models]

    if whisper_models:
        print("\n Whisper Models (use with --whisper):")
        for model in whisper_models:
            size_mb = os.path.getsize(os.path.join(MODELS_DIR, model)) / (1024 * 1024)
            print(f"    {model} ({size_mb:.1f} MB)")
    if llama_models:
```

```

    print("\n Llama Models (use with --llama):")
    for model in llama_models:
        size_gb=os.path.getsize(os.path.join(MODELS_DIR, model)) / (1024*1024*1024)
        print(f"    {model} ({size_gb:.2f} GB)")

    if other_files:
        print("\n📁 Other files:")
        for f in other_files:
            print(f"    {f}")

    print("\n Usage examples:")
    print(f"    python3 {sys.argv[0]} --whisper {whisper_models[0]} if whisper_models
else 'ggml-tiny.en.bin'}")
    print(f"    python3 {sys.argv[0]} --llama {llama_models[0]} if llama_models else
'model.gguf'}")
    print(f"    python3 {sys.argv[0]} -w {whisper_models[1]} if len(whisper_models) > 1
else 'model.bin' -t 8")

def resolve_model_path(filename):
    """Resolve a model filename to its full path."""

    # If it's already an absolute path, use it directly
    if os.path.isabs(filename):
        if os.path.exists(filename):
            return filename
        else:
            print(f" Model not found: {filename}")
            sys.exit(1)

    # Check in models directory
    model_path = os.path.join(MODELS_DIR, filename)
    if os.path.exists(model_path):
        return model_path
    # Check in current directory
    if os.path.exists(filename):
        return os.path.abspath(filename)
    print(f" Model '{filename}' not found in:")
    print(f"    - Current directory: {os.path.abspath(filename)}")
    print(f"    - Models directory: {model_path}")
    print(f"\n Tip: Use --list-models to see available models")
    sys.exit(1)

def run_voice_assistant(whisper_model, llama_model, threads):
    """Launch the complete voice assistant pipeline."""
    whisper_path = resolve_model_path(whisper_model)
    llama_path = resolve_model_path(llama_model)
    # Build the command
    cmd = [
        WHISPER_TALK_PATH,
        "-mw", whisper_path,
        "-ml", llama_path,
        "-t", str(threads),
        "--speak", "./speak"
    ]

    print("\n Starting Voice Assistant")
    print("=" * 60)
    print(f"    Whisper model: {os.path.basename(whisper_path)}")
    print(f"    Llama model:    {os.path.basename(llama_path)}")
    print(f"    CPU threads:    {threads}")
    print(f"    TTS engine:     Piper (female.onnx)")
    print("=" * 60)
    print("Speak into your microphone. The AI will respond with voice.")
    print("Press Ctrl+C to exit.\n")
    try:
        process = subprocess.run(
            cmd,
            stderr=subprocess.DEVNULL, # Hide ONNX Runtime warnings (2>/dev/null)
            cwd=os.path.expanduser("~/whisper.cpp")

```

```

    )
    sys.exit(process.returncode)

except KeyboardInterrupt:
    print("\n\n Voice assistant stopped.")
    sys.exit(0)

except FileNotFoundError:
    print(f" ERROR: {WHISPER_TALK_PATH} not found!")
    print("    Make sure whisper-talk-llama is built correctly.")
    print(f"    Expected location: {WHISPER_TALK_PATH}")
    sys.exit(1)

def main():
    """Main entry point."""
    args = parse_arguments()
    # Handle --list-models flag
    if args.list_models:
        list_available_models()
        sys.exit(0)
    # Validate thread count
    if args.threads < 1:
        print(" Thread count must be at least 1")
        sys.exit(1)
    if args.threads > 16:
        print(" Warning: High thread count may cause performance issues")
    # Run the voice assistant
    run_voice_assistant(args.whisper, args.llama, args.threads)

if __name__ == "__main__":
    main()

```

Sending transcribed text to MQTT server

```
#!/usr/bin/env python3
import subprocess
import socket
import argparse
import signal
import sys
import re
import random
import time
from datetime import datetime
# Paho MQTT client is required: pip install paho-mqtt
from paho.mqtt import client as mqtt_client
# --- Configuration ---
DEFAULT_MQTT_BROKER = "broker.emqx.io" # Free public MQTT broker
DEFAULT_MQTT_PORT = 1883
DEFAULT_TOPIC = "whisper/transcription"

def clean_transcription(text):
    # Remove IP addresses in brackets: [192.168.1.100]
    text = re.sub(r'\[\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\]', '', text)
    # Remove [BLANK_AUDIO] tag
    text = re.sub(r'\[BLANK_AUDIO\]', '', text, flags=re.IGNORECASE)
    # Remove action descriptions in brackets: [Sighs], [Laughs], [Music], etc.
    text = re.sub(r'\[[A-Za-z\s]+\]', '', text)
    # Remove parenthetical comments like (dramatic music), (applause), etc.
    text = re.sub(r'\([^\)]*\)', '', text)
    # Remove any remaining empty brackets
    text = re.sub(r'\[\]', '', text)
    # Clean up extra whitespace
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def connect_mqtt(broker, port, client_id):
    # Callback when client connects to broker
    def on_connect(client, userdata, flags, reason_code, properties):
        if reason_code == 0:
            print("Connected to MQTT Broker!")
        else:
            print("Failed to connect, return code %d\n" % reason_code)

    client = mqtt_client.Client(
        callback_api_version=mqtt_client.CallbackAPIVersion.VERSION2,
        client_id=client_id
    )
    client.on_connect = on_connect
    client.connect(broker, port)
    return client

def main():
    parser = argparse.ArgumentParser(description="Send whisper.cpp transcriptions via MQTT")
    parser.add_argument("--broker", default=DEFAULT_MQTT_BROKER,
                        help="MQTT broker address (default: %(default)s)")
    parser.add_argument("--port", type=int, default=DEFAULT_MQTT_PORT,
                        help="MQTT broker port (default: %(default)s)")
    parser.add_argument("--topic", default=DEFAULT_TOPIC,
                        help="MQTT topic to publish to (default: %(default)s)")
    parser.add_argument("--model", default="./models/ggml-tiny.en.bin",
                        help="Whisper model path")
    parser.add_argument("--threads", "-t", type=int, default=6,
                        help="Number of threads")
    parser.add_argument("--step", type=int, default=2048,
                        help="Step size in ms")
    parser.add_argument("--length", type=int, default=2048,
```

```

        help="Audio length in ms")
parser.add_argument("--whisper-path", default="./build/bin/whisper-stream",
                    help="Path to whisper-stream executable")
parser.add_argument("--no-filter", action="store_true",
                    help="Disable content filtering (send raw output)")
args = parser.parse_args()
# Generate a random client ID for MQTT connection
client_id = f'whisper-mqtt-{random.randint(0, 1000)}'
print("=" * 60)
print("Whisper MQTT Sender")
print("  MQTT Broker: " + args.broker + ":" + str(args.port))
print("  Topic: " + args.topic)
print("  Model: " + args.model)
print("  Threads: " + str(args.threads))
print("  Step/Length: " + str(args.step) + "ms / " + str(args.length) + "ms")
print("  Filtering: " + ("Disabled" if args.no_filter else "Enabled"))
print("=" * 60)
# Connect to MQTT broker
print("\nConnecting to MQTT broker...")
mqtt_client_obj = connect_mqtt(args.broker, args.port, client_id)
mqtt_client_obj.loop_start()
# Whisper-stream command
cmd = [
    args.whisper_path,
    "-m", args.model,
    "-t", str(args.threads),
    "--step", str(args.step),
    "--length", str(args.length)
]
print("Running: " + " ".join(cmd) + "\n")
# Flag to track if cleanup has already run
cleanup_done = [False]
# Cleanup handler
def cleanup(signum, frame):
    if cleanup_done[0]:
        return
    cleanup_done[0] = True
    print("\nStopping...")
    if process and process.poll() is None:
        process.terminate()
    mqtt_client_obj.loop_stop()
    mqtt_client_obj.disconnect()
    sys.exit(0)

signal.signal(signal.SIGINT, cleanup)
signal.signal(signal.SIGTERM, cleanup)
# Launch whisper-stream
try:
    process = subprocess.Popen(
        cmd,
        stdout=subprocess.PIPE,
        stderr=subprocess.DEVNULL,
        text=True,
        bufsize=1
    )
except FileNotFoundError:
    print("Error: " + args.whisper_path + " not found. Build whisper.cpp with
SDL2 support first.")
    mqtt_client_obj.loop_stop()
    mqtt_client_obj.disconnect()
    sys.exit(1)

print("Listening... Speak into the microphone (Ctrl+C to stop)\n")
# Read lines, filter, display locally, and publish via MQTT
for line in process.stdout:
    line = line.strip()
    if line:
        # Strip ANSI escape codes
        line = re.sub(r'\x1b\[([0-9;]*)[a-zA-Z]', '', line)

```

```

# Apply filters (unless disabled)
if args.no_filter:
    clean_line = line
else:
    clean_line = clean_transcription(line)
# Only print and send if there's content after filtering
if clean_line:
    timestamp = datetime.now().strftime("%H:%M:%S")
    # Show both raw and filtered in terminal
    if line != clean_line:
        print(" [" + timestamp + "] Raw: " + line)
        print(" [" + timestamp + "] Filtered: " + clean_line)
    else:
        print(" [" + timestamp + "] " + clean_line)
    # Publish the filtered version via MQTT
    result = mqtt_client_obj.publish(args.topic, clean_line)
    status = result[0]
    if status != 0:
        print(" [" + timestamp + "] Failed to send message to MQTT topic "
+ args.topic)

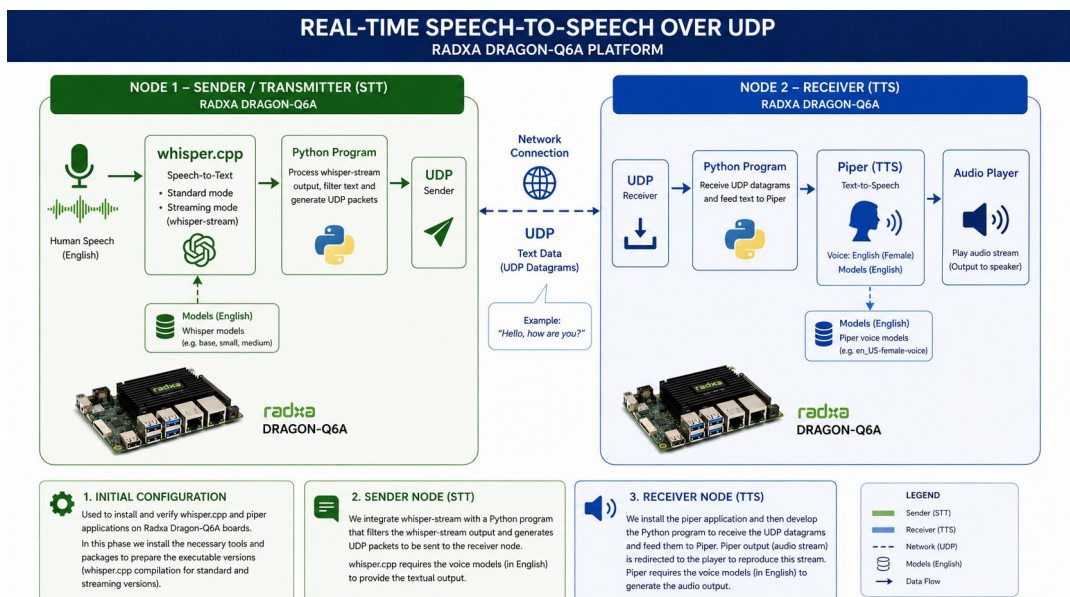
    cleanup(None, None)

if __name__ == "__main__":
    main()

```

Projects

1. Distributed Speech-to-Speech System using Radxa Dragon A6Q, whisper.cpp, UDP, and piper TTS



This configuration implements a **distributed speech communication system** based on two **Radxa Dragon A6Q Single Board Computers (SBCs)**. The first board performs **speech recognition** using **whisper.cpp** and transmits processed text messages over a network using **UDP packets**. The second Radxa Dragon A6Q board receives the text data and converts it into natural spoken audio using **Piper Text-to-Speech (TTS)** with a **female voice model**. The system enables low-latency speech transmission and audio regeneration across devices while operating fully on local hardware.

1. Speech Capture and Text Generation on the Sender Board

The first **Radxa Dragon A6Q SBC** acts as the **speech processing and transmission node**. A microphone connected to the board captures spoken speech from the user in real time. The captured audio is processed locally using **whisper.cpp**, an optimized implementation of automatic speech recognition (ASR) suitable for embedded systems.

The purpose of **whisper.cpp** is to transcribe spoken language into text. After speech recognition, the generated text undergoes a **filtering and cleanup stage** to improve readability and communication efficiency. This stage may remove repeated words, speech artifacts, filler expressions, low-confidence fragments, or unnecessary punctuation.

Example:

Speech input:

“Hello... hello everyone, welcome to the system demonstration.”

Filtered transcription output:

“Hello everyone welcome to the system demonstration”

The resulting filtered text is packaged into a **UDP data packet** and transmitted over the local network to the receiving board.

Operational mode of the sender board:

- Real-time microphone audio capture
- Local speech-to-text conversion using `whisper.cpp`
- Text filtering and cleanup
- UDP packet generation and transmission
- Continuous or event-triggered operation

2. UDP-Based Text Transmission

Communication between the two Radxa boards uses the **User Datagram Protocol (UDP)**. UDP is a lightweight transport protocol designed for fast message delivery with minimal communication overhead.

The sender board transmits filtered text messages as UDP payloads to the receiver board's IP address and designated listening port.

Example UDP payload:

“Welcome to the system demonstration”

Unlike TCP, UDP does not establish persistent connections or guarantee delivery, making it highly efficient for **real-time communication systems where low latency is more important than retransmission reliability**. In this application, occasional packet loss is generally acceptable because new speech segments continue arriving continuously.

Operational characteristics of UDP communication:

- Low-latency transmission
- Lightweight packet overhead
- No connection setup required
- Fast real-time message delivery
- Suitable for local network communication

3. Text Reception and Speech Synthesis on the Receiver Board

The second **Radxa Dragon A6Q SBC** functions as the **speech playback node**. This board continuously listens for incoming UDP packets on a predefined network port.

When a UDP packet containing text is received, the system extracts the text payload and forwards it to **Piper TTS**, a lightweight neural text-to-speech engine optimized for embedded and edge devices.

Piper converts the received text into natural-sounding speech using a **female voice model**.

Example:

Received text:

"Welcome to the system demonstration"

Generated audio output:

Natural synthesized female speech played through speakers or headphones.

The synthesized audio is then delivered to an external speaker, amplifier, or headphone output connected to the Radxa board.

Operational mode of the receiver board:

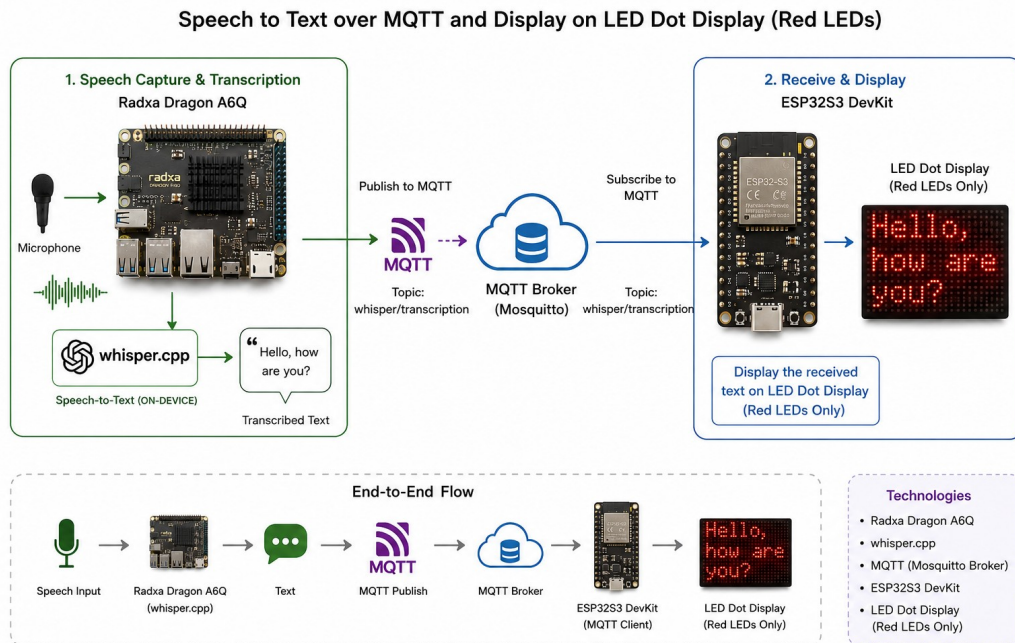
- UDP packet listening and reception
- Text extraction from UDP payloads
- Text-to-speech synthesis using Piper
- Female voice generation
- Real-time audio playback

End-to-End Operational Principles

The system follows a distributed speech-processing workflow:

1. **The user speaks into the microphone** connected to the sender Radxa Dragon A6Q.
2. **whisper.cpp converts speech into text** locally on the sender board.
3. **Text filtering removes unwanted fragments** and improves readability.
4. The sender board **encapsulates the filtered text into UDP packets** and transmits them across the network.
5. The receiver Radxa Dragon A6Q **receives the UDP packets** and extracts the text content.
6. **Piper TTS converts the text into speech** using a female voice model.
7. The synthesized speech is **played through an audio output device**.

2. Speech-to-Text MQTT Display System using Radxa Dragon A6Q and ESP32S3 IoT DevKit



This configuration implements a **distributed speech transcription and display system** using a **Radxa Dragon A6Q Single Board Computer (SBC)** and an **ESP32S3-based IoT DevKit**.

The system captures spoken speech, converts it into text using **whisper.cpp**, filters the transcription, and publishes the resulting text to an **MQTT broker**.

An ESP32S3 IoT device subscribes to the MQTT topic and displays the received text on a **red LED dot matrix display** in real time.

The architecture separates **speech processing** from **visual display**, allowing efficient operation across embedded devices connected over a network.

2.1 Speech Capture and Processing on Radxa Dragon A6Q

The system starts with a microphone connected to the **Radxa Dragon A6Q SBC**. Spoken audio from the user is captured and processed locally using **whisper.cpp**, a lightweight implementation of the Whisper speech recognition engine optimized for embedded systems.

The purpose of **whisper.cpp** is to convert speech into text. Once transcription is completed, the generated text undergoes a **filtering stage** to remove unnecessary words, invalid phrases, noise

artifacts, repeated fragments, or low-confidence recognition results. This ensures that only meaningful and readable text messages are sent through the messaging system.

For example:

Speech input:

`"Hello everyone, welcome to the demonstration."`

Filtered transcription output:

`"Hello everyone welcome to the demonstration"`

Operational mode of the Radxa SBC:

- Real-time speech capture from microphone
- Local speech recognition using `whisper.cpp`
- Text filtering and cleanup
- MQTT message publishing over Wi-Fi or Ethernet
- Continuous or event-driven processing

2.2 MQTT Message Distribution

After transcription and filtering, the Radxa Dragon A6Q publishes the text message to an **MQTT broker** using the topic:

`whisper/transcription`

MQTT acts as a lightweight messaging layer between devices. The broker receives published text messages and makes them available to subscribed clients.

Example MQTT payload:

Topic:

`whisper/transcription`

Message payload:

`"Welcome to the demonstration"`

The use of MQTT allows loose coupling between devices, enabling multiple display or automation systems to subscribe to the same transcription stream if needed.

Operational mode of MQTT:

- Publish/Subscribe communication model
- Lightweight network messaging
- Low bandwidth usage
- Real-time text distribution
- Reliable local network communication

3. ESP32S3 IoT DevKit Reception and Display

At the receiving end, an **ESP32S3-based IoT DevKit** subscribes to the MQTT topic `whisper/transcription`. The ESP32S3 continuously listens for incoming text messages from the MQTT broker.

When a new transcription message is received, the ESP32S3 processes the text and sends it to a **red LED dot matrix display**. The display presents the speech transcription visually, allowing nearby users to read spoken content in real time.

The display can operate in multiple modes, such as:

- Static text display
- Scrolling marquee text
- Line-by-line updates
- Automatic text refresh upon new MQTT messages

The red LED dot matrix provides high visibility and low power consumption, making it suitable for embedded systems, kiosks, assistive communication devices, and demonstration systems.

Operational mode of the IoT DevKit:

- Wi-Fi connection to MQTT broker
- Subscription to **whisper/transcription** topic
- MQTT message decoding
- Text rendering on red LED matrix
- Continuous listening and display updates

End-to-End Operation Steps

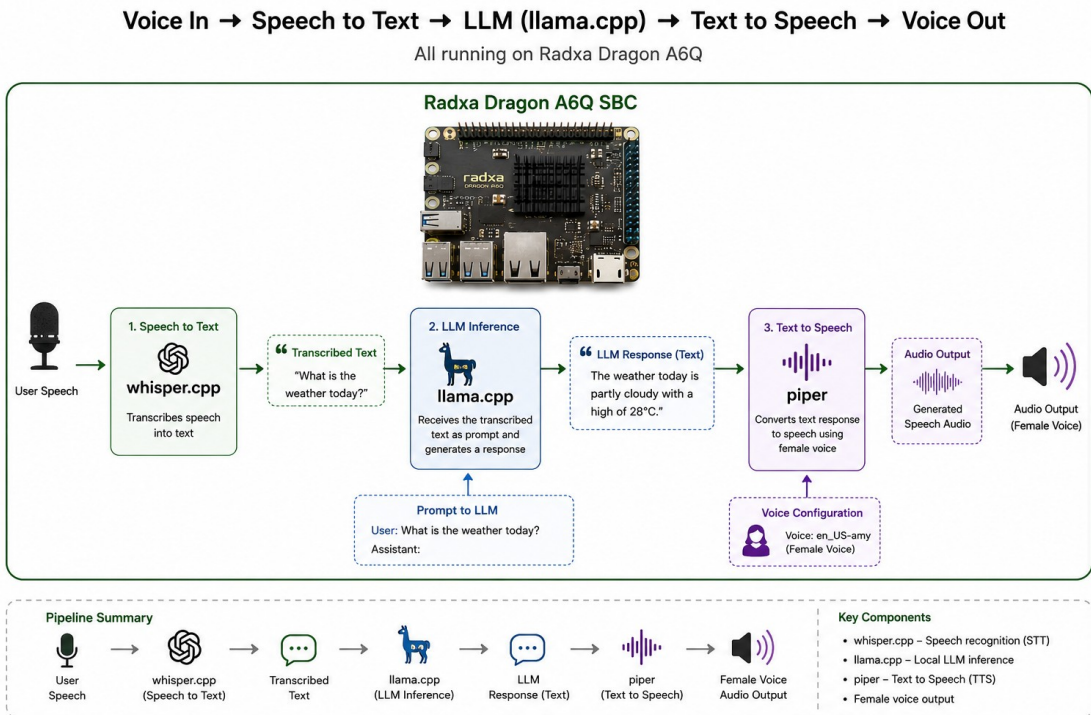
- **User speaks into the microphone** connected to the Radxa Dragon A6Q.
- **whisper.cpp transcribes speech into text** locally on the SBC.
- **Text filtering removes unwanted content** and improves readability.
- The Radxa board **publishes the filtered text** to the MQTT topic **whisper/transcription**.
- The **ESP32S3 IoT DevKit subscribes** to the MQTT topic and receives the message.
- The received text is **displayed on the red LED dot matrix** in real time.

Workflow Summary

Speech Input → Radxa Dragon A6Q → whisper.cpp → Filtered Text → MQTT Topic (whisper/transcription) → ESP32S3 IoT DevKit → Red LED Dot Matrix Display

This configuration creates a compact and efficient **real-time speech-to-text visualization system**, combining edge AI processing with IoT messaging and embedded display technology.

3. Audio assistant with `whisper.cpp`, `llama.cpp` and `piper`



This configuration implements a complete **local voice interaction pipeline** on a **Radxa Dragon A6Q single-board computer (SBC)**. The system captures spoken audio, converts it to text, processes the text through a local Large Language Model (LLM), and synthesizes a spoken response using text-to-speech (TTS).

All major processing stages run **on-device**, minimizing cloud dependency and improving privacy, latency, and offline capability.

1. Speech Input and Audio Capture

The process begins with a **microphone** connected to the Radxa Dragon A6Q board. The microphone captures the user's spoken speech in real time. Audio is streamed to the SBC through USB, audio jack, or an onboard audio interface depending on the hardware setup.

Operational mode:

- Continuous listening or push-to-talk mode
- Real-time audio streaming
- Mono PCM audio capture (typically 16 kHz for speech recognition)

2. Speech-to-Text using whisper.cpp

The captured audio is processed by **whisper.cpp**, a lightweight C/C++ implementation of OpenAI's Whisper automatic speech recognition (ASR) model optimized for edge devices.

Its role is to **transcribe spoken language into text**.

Example:

Input speech:

"What is the weather today?"

Transcribed text:

"What is the weather today?"

The transcription result becomes the **prompt input** for the language model.

Operational mode:

- Runs locally on CPU/NPU/GPU acceleration if available
- Real-time or near real-time inference
- Supports multilingual speech recognition
- Uses quantized Whisper models for reduced memory and faster execution

3. Local LLM Inference using llama.cpp

The text generated by whisper.cpp is sent to **llama.cpp**, a highly optimized inference engine for running **Large Language Models locally**.

The transcribed text is used as the **prompt message**, and the LLM generates a contextual response.

Example:

Prompt:

"What is the weather today?"

Generated response:

"Today is sunny with mild temperatures around 24 °C."

This stage provides the system's intelligence and conversational capability.

Operational mode:

- Fully local inference without internet
- Quantized models for efficient execution
- Token-by-token response generation
- Can operate in chat, assistant, or question-answering modes

4. Text-to-Speech using Piper TTS

The textual response from llama.cpp is then processed by **Piper TTS**, a fast and lightweight neural **text-to-speech engine**.

Piper converts the generated text into natural-sounding speech using a **female voice model**.

Example:

Input text:

“Today is sunny with mild temperatures around 24 °C.”

Generated output: Natural synthesized female speech.

Operational mode:

- Offline speech synthesis
- Neural voice generation
- Female voice profile (e.g., *en_US-amy*)
- Low-latency audio rendering

5. Audio Output

The synthesized voice is played back through a **speaker or headphone output** connected to the Radxa Dragon A6Q.

This completes the conversational loop:

Speech → Text → AI Reasoning → Speech

Operational mode:

- Real-time playback
- WAV/PCM audio streaming
- Speaker or amplifier output

End-to-End Workflow

User Speech → Microphone → whisper.cpp → Text Prompt → llama.cpp → Text Response → Piper TTS → Female Voice Audio Output

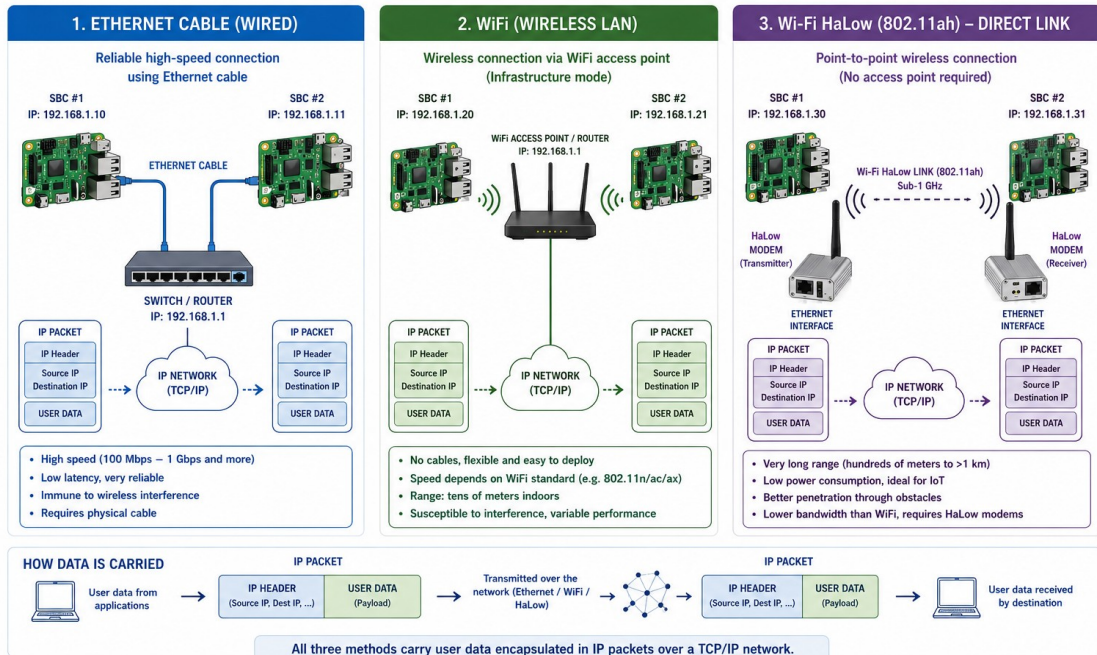
The entire pipeline runs **locally on the Radxa Dragon A6Q**, enabling a compact, privacy-preserving, low-latency AI voice assistant suitable for embedded systems, robotics, kiosks, smart devices, and offline conversational interfaces.

Interconnection means

When connecting Single Board Computers (SBCs) such as Radxa Dragon A6Q, Raspberry Pi, Jetson Nano, or similar embedded systems, there are several common methods for enabling communication over a local network.

The most widely used options are **wired Ethernet**, **Wi-Fi wireless networking**, and **Wi-Fi HaLow (IEEE 802.11ah) modems**, each offering different trade-offs in speed, range, power consumption, and deployment flexibility.

3 MEANS OF CONNECTION FOR SBCs TO CARRY USER DATA WITH IP PACKETS



Short overview of the three interconnection methods between two single-board computers (SBCs):

1. Ethernet cable (wired connection)

- Function:** Direct physical network link between SBCs via RJ45 ports.
- Capacity:** High throughput (typically 100 Mbps to 1 Gbps or more, depending on hardware).
- Strengths:** Very low latency, stable, secure, and immune to radio interference.
- Limitations:** Requires cabling; limited mobility and distance (usually up to 100 m per segment without repeaters).

2. Wi-Fi connection (wireless LAN)

- Function:** Wireless communication using standard WiFi (802.11 a/b/g/n/ac/ax).
- Capacity:** Variable (from ~20 Mbps to >1 Gbps in modern WiFi 5/6 setups under ideal conditions).

- **Strengths:** Flexible, no cables, easy deployment.
- **Limitations:** More latency and jitter than Ethernet, susceptible to interference, performance depends on distance and obstacles.

3. WiFi HaLow modems (IEEE 802.11ah) via Ethernet interface

- **Function:** Long-range, low-power wireless link using sub-1 GHz WiFi HaLow, typically bridged to SBC Ethernet ports via a modem/module.
- **Capacity:** Lower bandwidth than standard WiFi (typically hundreds of kbps up to several Mbps).
- **Strengths:** Very long range (hundreds of meters to over 1 km), good penetration through walls, low power consumption—ideal for IoT and remote SBC links.
- **Limitations:** Lower data rates compared to standard WiFi/Ethernet, less common hardware support.

In summary:

- Ethernet = fastest, most stable, wired backbone.
- WiFi = flexible, moderate-to-high speed, short/medium range.
- HaLow = long-range, low-power, lower-speed wireless bridge for distributed or remote SBC systems.